



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

DSP Lab - EE2801

Experiment-2

BY

Muthyala Koushik EE24BTECH11044

2026-02-08

Department of Electrical Engineering Indian Institute of Technology Hyderabad

Contents

1	Objective	1
2	Verilog Codes	2
3	MATLAB Code	5

1 Objective

To perform fixed-point addition, subtraction and multiplication on an array of data using Verilog with operands in Q(3,14) and Q(5,12) formats, and verify the outputs using MATLAB.

2 Verilog Codes

module

```

1 module fixed_point_ops (
2     input  signed [17:0] a_q314,    // Q(3,14)
3     input  signed [17:0] b_q512,    // Q(5,12)
4
5     output signed [20:0] sum_q614,  // Q(6,14)
6     output signed [20:0] sub_q614,  // Q(6,14)
7     output signed [35:0] mul_q926   // Q(9,26)
8 );
9
10    // ----- ADD/SUB ALIGNMENT -----
11
12    wire signed [20:0] a_q614;
13    assign a_q614 = {{3{a_q314[17]}}}, a_q314};
14
15    wire signed [20:0] b_q614;
16    assign b_q614 = {{1{b_q512[17]}}}, b_q512, 2'b00};
17
18    assign sum_q614 = a_q614 + b_q614;
19    assign sub_q614 = a_q614 - b_q614;
20
21    // ----- SHIFT-ADD MULTIPLIER -----
22
23    wire signed [35:0] a_ext;
24    wire signed [35:0] partial [17:0];
25    wire signed [35:0] mul_temp;
26
27    assign a_ext = {{18{a_q314[17]}}}, a_q314};
28
29    genvar i;
30    generate
31        for(i = 0; i < 17; i = i + 1) begin : MULT_STAGE
32            assign partial[i] = b_q512[i] ? (a_ext <<< i) : 36'sd0;
33        end
34    endgenerate
35
36    // sign bit handling

```

```

37 assign partial[17] = b_q512[17] ? -(a_ext <<< 17) : 36'sd0;
38
39 assign mul_temp =
40     partial[0] + partial[1] + partial[2] + partial[3] +
41     partial[4] + partial[5] + partial[6] + partial[7] +
42     partial[8] + partial[9] + partial[10] + partial[11] +
43     partial[12] + partial[13] + partial[14] + partial[15] +
44     partial[16] + partial[17];
45
46 assign mul_q926 = mul_temp;
47
48 endmodule

```

testbench

```

1 `timescale 1ns/1ps
2
3 module tb_fixed_point_ops;
4
5     // Full dataset for MATLAB verification
6     parameter N = 48000;
7
8     // memory for inputs
9     reg signed [17:0] x_q314_mem [0:N-1];
10    reg signed [17:0] x_q512_mem [0:N-1];
11
12    // DUT inputs
13    reg signed [17:0] a_q314;
14    reg signed [17:0] b_q512;
15
16    // DUT outputs
17    wire signed [20:0] sum_q614;
18    wire signed [20:0] sub_q614;
19    wire signed [35:0] mul_q926;
20
21    integer f_sum, f_sub, f_mul;
22    integer i;
23    integer dump_count;
24

```

```

25 // Instantiate DUT
26 fixed_point_ops DUT (
27     .a_q314(a_q314),
28     .b_q512(b_q512),
29     .sum_q614(sum_q614),
30     .sub_q614(sub_q614),
31     .mul_q926(mul_q926)
32 );
33
34 // ----- GTKWave Dump -----
35 initial begin
36     $dumpfile("fixed_point.vcd");
37     $dumpvars(0, tb_fixed_point_ops);
38     dump_count = 0;
39 end
40
41 initial begin
42
43     $display("Reading input files...");
44
45     // read inputs
46     $readmemb("x_q314.txt", x_q314_mem);
47     $readmemb("x_q512.txt", x_q512_mem);
48
49     // open output files
50     f_sum = $fopen("sum_q614.txt", "w");
51     f_sub = $fopen("sub_q614.txt", "w");
52     f_mul = $fopen("mul_q926.txt", "w");
53
54     // initialize inputs
55     a_q314 = 0;
56     b_q512 = 0;
57     #10;
58
59     // apply stimulus
60     for (i = 0; i < N; i = i + 1) begin
61
62         a_q314 = x_q314_mem[i];
63         b_q512 = x_q512_mem[i];
64

```

```

65         #10;
66
67         // write outputs for MATLAB
68         $fdisplay(f_sum, "%d", sum_q614);
69         $fdisplay(f_sub, "%d", sub_q614);
70         $fdisplay(f_mul, "%d", mul_q926);
71
72         // limit GTKWave dumping to first 5 cycles
73         dump_count = dump_count + 1;
74         if (dump_count == 240)
75             $dumpoff;
76     end
77
78     // close files
79     $fclose(f_sum);
80     $fclose(f_sub);
81     $fclose(f_mul);
82
83     $display("Simulation complete. Output files generated.");
84     $finish;
85 end
86
87 endmodule

```

3 MATLAB Code

```

1  clc; clear; close all;
2
3  %% Parameters
4  fs = 48000;
5  f  = 1000;
6  Ns = 5 * fs / f;    % 240 samples
7
8  %% Read Verilog outputs and convert to real
9  ver_add = load('sum_q614.txt') / 2^14;
10 ver_sub = load('sub_q614.txt') / 2^14;
11 ver_mul = load('mul_q926.txt') / 2^26;
12
13 ver_add = ver_add(1:Ns);

```

```

14 ver_sub = ver_sub(1:Ns);
15 ver_mul = ver_mul(1:Ns);
16
17 %% MATLAB reference computation
18 t = (0:Ns-1)/fs;
19 x = 2*sin(2*pi*f*t);
20
21 % Q(3,14)
22 op1_int = round(x * 2^14);
23 op1_real = op1_int / 2^14;
24
25 % Q(5,12)
26 op2_int = round(x * 2^12);
27 op2_real = op2_int / 2^12;
28
29 % Align fractional bits
30 op2_align = op2_real * 2^2;
31
32 % Reference results
33 add_ref = op1_real + op2_align;
34 sub_ref = op1_real - op2_align;
35 mul_ref = op1_real .* op2_real;
36
37 %% Plot Addition
38 figure;
39 plot(add_ref, 'b'); hold on;
40 plot(ver_add, 'r—');
41 grid on;
42 legend('MATLAB Reference', 'Verilog Output');
43 title('Fixed-Point Addition (5 Cycles)');
44
45 %% Plot Subtraction
46 figure;
47 plot(sub_ref, 'b'); hold on;
48 plot(ver_sub, 'r—');
49 grid on;
50 legend('MATLAB Reference', 'Verilog Output');
51 title('Fixed-Point Subtraction (5 Cycles)');
52
53 %% Plot Multiplication

```



```
54 figure;  
55 plot(mul_ref, 'b'); hold on;  
56 plot(ver_mul, 'r—');  
57 grid on;  
58 legend('MATLAB Reference', 'Verilog Output');  
59 title('Fixed-Point Multiplication (5 Cycles)');
```

Verilog Code Explanation

The Verilog module implements fixed-point addition, subtraction, and multiplication using operands in Q(3,14) and Q(5,12) formats.

For addition and subtraction, both operands are aligned to the same fractional precision by converting them into Q(6,14). This is achieved by sign-extending the Q(3,14) input and left-shifting the Q(5,12) input by two bits. The aligned values are then added and subtracted to obtain the outputs.

Multiplication is implemented using a shift-add technique. The multiplicand is sign-extended, and partial products are generated based on each bit of the multiplier. The sign bit of the multiplier is handled using two's complement subtraction. All partial products are accumulated to produce the final multiplication result in Q(9,26) format.

The testbench reads operand data from files generated in MATLAB, applies them to the Verilog module, and writes the computed addition, subtraction, and multiplication outputs to text files. Waveforms are also generated for a limited number of cycles for visualization using GTKWave.

MATLAB Code Explanation

MATLAB is used to verify the outputs obtained from the Verilog implementation. The output files generated by Verilog for addition, subtraction, and multiplication are read and converted from fixed-point format to floating-point values using appropriate scaling factors.

A sine-wave signal is generated and quantized into Q(3,14) and Q(5,12) formats. These quantized values are used to compute reference results for addition, subtraction, and multiplication after aligning the fractional bits.

Only the first five cycles of data are considered for analysis and plotting. The MATLAB reference outputs and Verilog outputs are plotted together, and the close match between them confirms the correctness of the fixed-point arithmetic implementation.

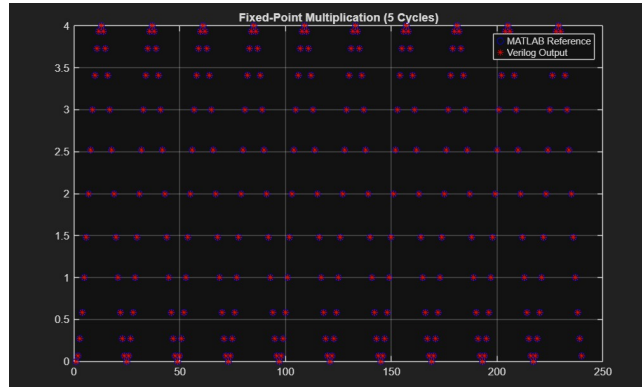


Figure 1: Enter Caption

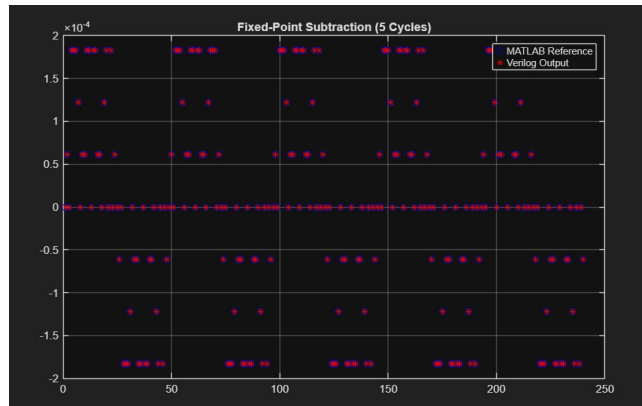


Figure 2: Enter Caption

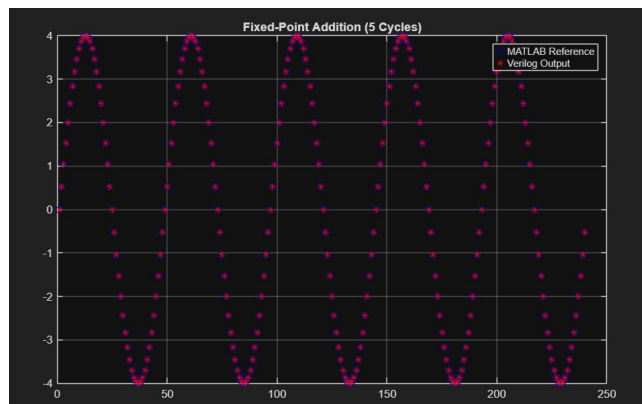


Figure 3: Enter Caption